

Egos-2000 Documentation Group-66

Steps to run on Ubuntu:

1. Open 2 different Ubuntu terminals
2. On terminal 1: `docker start -ai 7f4521829440`
3. Then on terminal 2:
`docker exec -it 7f4521829440 rm -rf /root/egos/egos-2000`
`docker cp /mnt/c/Users/user/Desktop/egos-2000 7f4521829440:/root/egos/`
4. Then back on terminal 1:
`cd /root/egos/egos-2000`
`make`
`make qemu`
`0`
5. Then normally run `grep` and `wcl` commands(eg: `grep lines README`, `wcl README`)
6. Process lifecycle stats should be displayed

Implementation:

- Implemented `grep` and `wcl` commands (added files for both in apps -> user)
- `grep` - reads the entire file and processes it block by block till EOF, truncates line if too long, check for pattern using `strstr()` to find pattern
- `wcl` - processes each file, reads the entire file and processes it block by block till EOF, counts the number of newlines in each block
- Implemented MLFQ scheduling by doing changes in `process.h`, `process.c` and `kernel.c`
- `process.h` : added fields for lifecycle stats mlfq scheduling
- `process.c` : changes in `proc_alloc()`, `proc_free()`, `mlfq_update_level()`, `mlfq_reset_level()`
- `proc_alloc()` - initializes all fields when a process is created and sets priority level to 0
- `proc_free()` - calculates and prints process lifecycle time stats before freeing a process
- `mlfq_update_level()` - called after each timer interrupt to update process MLFQ level, if process exceeds its time quantum for current level, it is demoted
- `mlfq_reset_level()` - all processes reset to Level 0 after MLFQ_RESET_PERIOD and if user pressed a key, finds GPID_SHELL and resets it to Level 0 (shell immediately responds to user input)
- `kernel.c` : changes in `intr_entry()`, `proc_yield()`
- `intr_entry` - timer interrupt handler called on every timer interrupt, increments `timer_interrupt_count` for the process, updates CPU time (assuming 1ms per timer interrupt), calls `mlfq_update_level()` to potentially demote the process
- `proc_yield()` - calls `mlfq_reset_level()` first to check for keyboard input and periodic reset, searches for next runnable process level by level starting with the highest level 0, records `first_run_time` when process executes the first time
- Proper error handling performed wherever needed in `grep`, `wcl`, `kernel.c`

Github repository link: <https://github.com/antelope0112/Group66-egos>